Lecture Scribe Guidelines

L10_S2.pdf
PDF

1. Purpose and Evaluation Context
This output is a graded academic artifact, not a creative task. The lecture scribe must be suitable for reading-based / closed-notes exam preparation and serve as faithful reference material for revising what was taught in class.
This task evaluates: Precision in prompt adherence; Faithfulness to provided material; Disciplined and reproducible use of AI.
This task does not evaluate: Creativity; Personal writing style; Fluency or verbosity beyond necessity.
The scribe must answer the question: "If a student studies only this document, can they reliably revise what was taught?"

2. Definition of a Lecture Scribe
A lecture scribe is a faithful, exam-ready reconstruction of lecture content. A correct lecture scribe must allow a student to: Recall exact definitions and notation; Reconstruct proofs and derivations; Follow worked examples step by step; Understand logical dependencies between concepts.

3. Context Constraint (Authoritative Sources Only)
The only allowed context consists of: The attached Lecture slides / lecture PDF; Attached textbook excerpts that are explicitly relevant to this lecture. You must not use any other source of information.
Not Allowed: Solved material not shown or discussed in lecture; Instructor meta-instructions on how to write a scribe; Reference scribes or ideal answers; Writing guides or explanations disguised as content; Intuition, commentary, or simplifications not present in lecture.
Everything included in the scribe must be directly traceable to the provided context.

4. Content Requirements
The lecture scribe must include, exactly as presented in the lecture: Definitions and notation; Assumptions and conditions; Statements of theorems or results; Proofs or proof sketches, reconstructed step by step; Worked examples only if they appear in the context, with all intermediate steps; Clear logical flow following the lecture's structure and order.
You must not: Invent definitions, results, notation, or examples; Add explanations or intuition not explicitly discussed; Reorder topics beyond the lecture's flow.

5. Reasoning Requirement (Chain-of-Thought)
You must include explicit reasoning steps in the scribe wherever the lecture includes: Proofs; Derivations; Example solutions.
Each reasoning step must follow only from: Previously stated definitions; Assumptions or conditions; Earlier results from the same lecture.
Reasoning must reflect how conclusions follow, not external intuition.

6. Structure Requirement
Organize the scribe using clear sectioning that mirrors the lecture structure (for example): Random Variables; Bernoulli Random Variable; Binomial Distribution; Geometric Distribution; Poisson Distribution.
Logical dependencies between concepts must be explicit.

7. Output Constraint
Output only the lecture scribe. No commentary before or after the scribe. No meta explanations about the process.

8. Self-Consistency Prompting Requirement
This prompt is designed for self-consistency evaluation. You must assume that: The same prompt will be reused across multiple independent runs; The same context will be reused across all runs. Your task in each run is to generate one complete lecture scribe, fully self-contained. No reference to other runs is allowed.

9. Instruction to Begin
Generate the lecture scribe now, strictly following all constraints above.

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

Instructor: Dhaval Patel, PhD
Date: February 5, 2026

---

1. Min-Cut Problem

1.1 Why Use Min-Cut?

We use the min-cut algorithm in various applications to solve problems related to **network connectivity, reliability, and optimization** L10_S2.

Applications include:

- **Network Design:**
  Min-cut helps in improving the efficiency of communication and optimizing network flow. The algorithm is used in network design to find the minimum capacity cut L10_S2.
- **Communication Networks:**
  For understanding the vulnerability of networks to failures, minimum cut can be useful. It helps build robust and fault-tolerant communication networks L10_S2.
- **VLSI Design:**
  In Very Large Scale Integration (VLSI) design, the algorithm is useful for partitioning circuits into smaller components, leading to reduced interconnectivity complexity L10_S2.

Reference is made to Section 1.5: *Application: A Randomized Min-Cut Algorithm* in *Probability and Computing (2nd Edition)* L10_S2.

1.2 What is Min-Cut?

A **cut-set** in a graph is a set of edges whose removal breaks the graph into two or more connected components [L10_S2].

Given a graph

$$G = (V, E)$$

with n vertices, the **minimum cut (min-cut) problem** is to find a **minimum cardinality cut-set** in G [L10_S2].

Min-cut algorithms, such as **Karger's algorithm**, are random and can be sensitive to the initial choice of edges. If the algorithm happens to contract critical edges early in the process, it might find a smaller cut [L10_S2].

1.3 Edge Contraction

The main operation in the algorithm is **edge contraction** [L10_S2].

Edge contraction is an operation that removes an edge from a graph while simultaneously merging the two vertices [L10_S2].

In contracting an edge (u, v):

1. Merge the two vertices u and v into one vertex.
2. Eliminate all edges connecting u and v.
3. Retain all other edges in the graph.
4. The new graph may have parallel edges but no self-loops [L10_S2].

---

2. Successful and Unsuccessful Min-Cut Runs

2.1 Successful Min-Cut Run

A **successful min-cut run** refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph [L10_S2].

The figure in the lecture illustrates a sequence of edge contractions that results in the correct minimum cut (Figure: Successful Run) [L10_S2].

2.2 Unsuccessful Min-Cut Run

An **unsuccessful min-cut run** refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph [L10_S2].

The figure in the lecture illustrates contractions that lead to a non-minimum cut (Figure: Unsuccessful Run) [L10_S2].

---

3. Max-Flow Min-Cut Theorem

The **Max-Flow Min-Cut Theorem** states:

> "In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut." L10_S2

Definitions:

- **Capacity of a cut:**
  The sum of the capacity of the edges in the cut that are oriented from a vertex $\in X$ to a vertex $\in Y$ L10_S2.
- **Minimum cut:**
  The cut in the network that has the smallest possible capacity L10_S2.
- **Minimum cut capacity:**
  The capacity of the minimum cut L10_S2.
- **Maximum flow:**
  The largest possible flow from source S to sink T L10_S2.

---

4. Deterministic Min-Cut Algorithm

4.1 Stoer–Wagner Min-Cut Algorithm

Let s and t be two vertices of a graph G. Let $G/\{s,t\}$ be the graph obtained by merging s and t.

Then a minimum cut of G can be obtained by taking the smaller of:

1. A minimum s-t-cut of G, and
2. A minimum cut of $G/\{s,t\}$ L10_S2.

The theorem holds since:

- Either there is a minimum cut of G that separates s and t, then a minimum s-t-cut of G is a minimum cut of G;
- Or there is none, then a minimum cut of $G/\{s,t\}$ does the job L10_S2.

4.2 Pseudocode

Algorithm 1: MinimumCutPhase(G, a) L10_S2

1. $A \leftarrow \{a\}$
2. while $A \neq V$ do
   - add to A the most tightly connected vertex
3. return the cut weight as the "cut of the phase"

Algorithm 2: MinimumCut(G) L10_S2

1. while $|V| \geq 1$ do
    - choose any a from V;
    - MinimumCutPhase(G, a);
    - if the cut-of-the-phase is lighter than the current minimum cut then
        - store the cut-of-the-phase as the current minimum cut;
    - shrink G by merging the two vertices added last;
2. return the minimum cut

---

5. Randomized Min-Cut Algorithm

5.1 Why Randomized Algorithm?

Randomized algorithms provide a probabilistic guarantee of success. They provide a more accurate estimate of the minimum cut with fewer iterations L10_S2.

Characteristics listed in lecture:

- Efficiency
- Parallelization
- Approximation Guarantees
- Avoidance of Worst-Case Instances
- Heuristic Nature
- Robustness L10_S2

5.2 Karger's Randomized Algorithm

The lecture provides an example run where random edges are picked in such a way that the output cut is not minimal. The output cut produced is {a, c, e}, but the minimal cut is either {b, e} or {a, d} L10_S2.

This illustrates the possibility of unsuccessful runs due to random contraction choices.

5.3 Pseudocode: Recursive-Randomized-Min-Cut(G, α)

Input:
An undirected multigraph G with n vertices and an integer constant $\alpha > 0$.

Output:
A cut C of G. L10_S2

If $n \leq \alpha^3$ then:

- $C \leftarrow$ a min-cut of G found using brute force (exhaustive) search;

Else:

- for $i \leftarrow 1$ to α do
  - $G' \leftarrow$ multigraph obtained by applying $n - \frac{n}{\sqrt{\alpha}}$ random contraction steps on G;
  - $C' \leftarrow$ Recursive-Randomized-Min-Cut(G', α);
  - if $i = 1$ or $|C'| < |C|$ then
    - $C \leftarrow C'$;

Return C L10_S2.

---

6. Comparison: Deterministic vs Randomized Min-Cut

Any specific problem is responsible for deciding the approach L10_S2.

Deterministic Min Cut:

- Always guarantees an exact minimum cut.
- May have higher time complexity for large graphs.
- Stoer–Wagner algorithm has time complexity

$$O(V \cdot E + V^2 \log V)$$

  where V is number of vertices and E is number of edges L10_S2.

Randomized Min Cut:

- Provides an approximate minimum cut with high probability.
- Karger's algorithm has time complexity

$$O(V^2)$$

  where V is the number of vertices L10_S2.

---

7. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least

$$\frac{2}{n(n-1)}.$$

L10_S2

---

8. Python Simulation

- Open Campuswire Post regarding today's lecture (L10).
- Download the `.ipynb` file pasted in the comments L10_S2.

End of Lecture 10

Sources

1. Purpose and Evaluation Context
This output is a graded academic artifact, not a creative task. The lecture scribe must be suitable for reading-based / closed-notes exam preparation and serve as faithful reference material for revising what was taught in class.
This task evaluates: Precision in prompt adherence; Faithfulness to provided material; Disciplined and reproducible use of AI.
This task does not evaluate: Creativity; Personal writing style; Fluency or verbosity beyond necessity.
The scribe must answer the question: "If a student studies only this document, can they reliably revise what was taught?"

2. Definition of a Lecture Scribe
A lecture scribe is a faithful, exam-ready reconstruction of lecture content. A correct lecture scribe must allow a student to: Recall exact definitions and notation; Reconstruct proofs and derivations; Follow worked examples step by step; Understand logical dependencies between concepts.

3. Context Constraint (Authoritative Sources Only)
The only allowed context consists of: The attached Lecture slides / lecture PDF; Attached textbook excerpts that are explicitly relevant to this lecture. You must not use any other source of information.
Not Allowed: Solved material not shown or discussed in lecture; Instructor meta-instructions on how to write a scribe; Reference scribes or ideal answers; Writing guides or explanations disguised as content; Intuition, commentary, or simplifications not present in lecture.
Everything included in the scribe must be directly traceable to the provided context.

4. Content Requirements
The lecture scribe must include, exactly as presented in the lecture: Definitions and notation; Assumptions and conditions; Statements of theorems or results; Proofs or proof sketches, reconstructed step by step; Worked examples only if they appear in the context, with all intermediate steps; Clear logical flow following the lecture's structure and order.
You must not: Invent definitions, results, notation, or examples; Add explanations or intuition not explicitly discussed; Reorder topics beyond the lecture's flow.

5. Reasoning Requirement (Chain-of-Thought)
You must include explicit reasoning steps in the scribe wherever the lecture includes: Proofs; Derivations; Example solutions.
Each reasoning step must follow only from: Previously stated definitions; Assumptions or conditions; Earlier results from the same lecture.
Reasoning must reflect how conclusions follow, not external intuition.

6. Structure Requirement
Organize the scribe using clear sectioning that mirrors the lecture structure (for example): Random Variables; Bernoulli Random Variable; Binomial Distribution; Geometric Distribution; Poisson Distribution.
Logical dependencies between concepts must be explicit.

7. Output Constraint
Output only the lecture scribe. No commentary before or after the scribe. No meta explanations about the process.

8. Self-Consistency Prompting Requirement
This prompt is designed for self-consistency evaluation. You must assume that: The same prompt will be reused across multiple independent runs; The same context will be reused across all runs. Your task in each run is to generate one complete lecture scribe, fully self-contained. No reference to other runs is allowed.

9. Instruction to Begin
Generate the lecture scribe now, strictly following all constraints above.

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

Instructor: Dhaval Patel, PhD
Date: February 5, 2026

---

1. Min-Cut Problem

1.1 Why Use Min-Cut?

We use the min-cut algorithm in various applications to solve problems related to **network connectivity, reliability, and optimization** L10_S2.

Applications listed in the lecture:

- **Network Design:**
  Min cut helps in improving the efficiency of communication and optimizing network flow. The algorithm is used in network design to find the minimum capacity cut L10_S2.
- **Communication Networks:**
  For understanding the vulnerability of the networks to failures minimum cut can be useful. It helps building robust and fault-tolerant communication networks L10_S2.
- **VLSI Design:**
  In Very Large Scale Integration (VLSI) design, the algorithm is useful for partitioning circuits into smaller components leading to reduced interconnectivity complexity L10_S2.

Reference: Section 1.5, *Application: A Randomized Min-Cut Algorithm* in *Probability and Computing (2nd Edition)* L10_S2.

1.2 What is Min-Cut?

A **cut-set** in a graph is a set of edges whose removal breaks the graph into two or more connected components [L10_S2].

Given a graph

$$G = (V, E)$$

with n vertices, the **minimum cut (min-cut) problem** is to find a **minimum cardinality cut-set** in G [L10_S2].

Min-cut algorithms, such as Karger's algorithm, are random and can be sensitive to the initial choice of edges. If the algorithm happens to contract critical edges early in the process, it might find a smaller cut [L10_S2].

1.3 Edge Contraction

The main operation in the algorithm is **edge contraction**, which is operation that removes an edge from a graph while simultaneously merging the two vertices [L10_S2].

In contracting an edge (u, v):

1. Merge the two vertices u and v into one vertex.
2. Eliminate all edges connecting u and v.
3. Retain all other edges in the graph.
4. The new graph may have parallel edges but no self-loops [L10_S2].

---

2. Successful and Unsuccessful Min-Cut Runs

2.1 Successful Min-Cut Run

A successful min cut run refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph [L10_S2].

The lecture includes a figure titled **"Figure: Successful Run"** illustrating contractions that result in the correct minimum cut [L10_S2].

2.2 Unsuccessful Min-Cut Run

An unsuccessful min-cut run refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph [L10_S2].

The lecture includes a figure titled **"Figure: Unsuccessful Run"** illustrating contractions that lead to a non-minimum cut [L10_S2].

---

3. Max-Flow Min-Cut Theorem

The **Max-Flow Min-Cut Theorem** states:

> "In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut." L10_S2

Definitions provided in the lecture:

- **Capacity of a cut:**
  The sum of the capacity of the edges in the cut that are oriented from a vertex $\in X$ to a vertex $\in Y$ L10_S2.
- **Minimum cut:**
  The cut in the network that has the smallest possible capacity L10_S2.
- **Minimum cut capacity:**
  The capacity of the minimum cut L10_S2.
- **Maximum flow:**
  The largest possible flow from source S to sink T L10_S2.

---

4. Deterministic Min-Cut Algorithm

4.1 Stoer–Wagner Min-Cut Algorithm

Let s and t be two vertices of a graph G. Let $G/\{s,t\}$ be the graph obtained by merging s and t.

Then a minimum cut of G can be obtained by taking the smaller of:

1. A minimum s-t-cut of G, and
2. A minimum cut of $G/\{s,t\}$ L10_S2.

Reasoning as presented in lecture:

- Either there is a minimum cut of G that separates s and t; then a minimum s-t-cut of G is a minimum cut of G.
- Or there is none; then a minimum cut of $G/\{s,t\}$ does the job L10_S2.

4.2 Pseudocode

Algorithm 1: MinimumCutPhase(G, a) L10_S2

1. $A \leftarrow \{a\}$
2. while $A \neq V$ do
   - add to A the most tightly connected vertex
3. return the cut weight as the "cut of the phase"

Algorithm 2: MinimumCut(G) L10_S2

1. while $|V| \geq 1$ do

- choose any a from V;
- MinimumCutPhase(G, a);
- if the cut-of-the-phase is lighter than the current minimum cut then
  - store the cut-of-the-phase as the current minimum cut;
- shrink G by merging the two vertices added last;

2. return the minimum cut

---

5. Randomized Min-Cut Algorithm

5.1 Why Randomized Algorithm?

Randomized algorithms provide a probabilistic guarantee of success. It provides a more accurate estimate of the minimum cut with fewer iterations L10_S2 .

Characteristics listed:

- Efficiency
- Parallelization
- Approximation Guarantees
- Avoidance of Worst-Case Instances
- Heuristic Nature
- Robustness L10_S2

5.2 Karger's Randomized Algorithm

The lecture provides an example run where random edges are picked such that the output cut is not minimal. The output cut produced is {a, c, e}, but the minimal cut is either {b, e} or {a, d} L10_S2 .

This demonstrates the possibility of unsuccessful runs due to random edge contractions.

5.3 Pseudocode: Recursive-Randomized-Min-Cut(G, α)

Input:
An undirected multigraph G with n vertices and an integer constant $\alpha > 0$.

Output:
A cut C of G L10_S2 .

If $n \leq \alpha^3$ then

- $C \leftarrow$ a min-cut of G found using brute force (exhaustive) search;

Else

- for $i \leftarrow 1$ to α do

- $G' \leftarrow$ multigraph obtained by applying $n - \frac{n}{\sqrt{\alpha}}$ random contraction steps on G;
- $C' \leftarrow$ Recursive-Randomized-Min-Cut(G', α);
- if $i = 1$ or $|C'| < |C|$ then
  - $C \leftarrow C'$;

Return C L10_S2.

---

6. Comparison: Deterministic vs Randomized Min-Cut

Any specific problem is responsible for deciding the approach L10_S2.

Deterministic Min Cut:

- Always guarantees an exact minimum cut.
- May have higher time complexity for large graphs.
- Stoer–Wagner algorithm has time complexity

$$O(V \cdot E + V^2 \log V)$$

  where V is the number of vertices and E is the number of edges L10_S2.

Randomized Min Cut:

- Provides an approximate minimum cut with high probability.
- Karger's algorithm has time complexity

$$O(V^2)$$

  where V is the number of vertices L10_S2.

---

7. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least

$$\frac{2}{n(n-1)}.$$

L10_S2

---

8. Python Simulation

- Open Campuswire Post regarding today's lecture (L10).
- Download the `.ipynb` file pasted in the comments L10_S2.

End of Lecture 10

Sources

1. Purpose and Evaluation Context
This output is a graded academic artifact, not a creative task. The lecture scribe must be suitable for reading-based / closed-notes exam preparation and serve as faithful reference material for revising what was taught in class.
This task evaluates: Precision in prompt adherence; Faithfulness to provided material; Disciplined and reproducible use of AI.
This task does not evaluate: Creativity; Personal writing style; Fluency or verbosity beyond necessity.
The scribe must answer the question: "If a student studies only this document, can they reliably revise what was taught?"

2. Definition of a Lecture Scribe
A lecture scribe is a faithful, exam-ready reconstruction of lecture content. A correct lecture scribe must allow a student to: Recall exact definitions and notation; Reconstruct proofs and derivations; Follow worked examples step by step; Understand logical dependencies between concepts.

3. Context Constraint (Authoritative Sources Only)
The only allowed context consists of: The attached Lecture slides / lecture PDF; Attached textbook excerpts that are explicitly relevant to this lecture. You must not use any other source of information.
Not Allowed: Solved material not shown or discussed in lecture; Instructor meta-instructions on how to write a scribe; Reference scribes or ideal answers; Writing guides or explanations disguised as content; Intuition, commentary, or simplifications not present in lecture.
Everything included in the scribe must be directly traceable to the provided context.

4. Content Requirements
The lecture scribe must include, exactly as presented in the lecture: Definitions and notation; Assumptions and conditions; Statements of theorems or results; Proofs or proof sketches, reconstructed step by step; Worked examples only if they appear in the context, with all intermediate steps; Clear logical flow following the lecture's structure and order.
You must not: Invent definitions, results, notation, or examples; Add explanations or intuition not explicitly discussed; Reorder topics beyond the lecture's flow.

5. Reasoning Requirement (Chain-of-Thought)
You must include explicit reasoning steps in the scribe wherever the lecture includes: Proofs; Derivations; Example solutions.
Each reasoning step must follow only from: Previously stated definitions; Assumptions or conditions; Earlier results from the same lecture.
Reasoning must reflect how conclusions follow, not external intuition.

6. Structure Requirement
Organize the scribe using clear sectioning that mirrors the lecture structure (for example): Random Variables; Bernoulli Random Variable; Binomial Distribution; Geometric Distribution; Poisson Distribution.
Logical dependencies between concepts must be explicit.

7. Output Constraint
Output only the lecture scribe. No commentary before or after the scribe. No meta explanations about the process.

8. Self-Consistency Prompting Requirement
This prompt is designed for self-consistency evaluation. You must assume that: The same prompt will be reused across multiple independent runs; The same context will be reused across all runs. Your task in each run is to generate one complete lecture scribe, fully self-contained. No reference to other runs is allowed.

9. Instruction to Begin
Generate the lecture scribe now, strictly following all constraints above.

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

Instructor: Dhaval Patel, PhD
Date: February 5, 2026

Outline

- Min-Cut Problem
- Why use min-cut?
- What is min-cut?
- Successful min-cut run
- Unsuccessful min-cut run
- Max-Flow Min-Cut Theorem
- Deterministic Min-Cut Algorithm
- Stoer–Wagner Minimum Cut Algorithm
- Pseudocode
- Randomized Min-Cut Algorithm
- Why randomized algorithm?
- Karger's Randomized Algorithm
- Pseudocode
- Comparison: Deterministic Min-Cut vs Randomized Min-Cut Algorithm
- Theorem for min-cut set
- Python Simulation
- Class Participation

1. Min-Cut Problem

1.1 Why Use Min-Cut?

We use the min-cut algorithm in various applications to solve problems related to network connectivity, reliability, and optimization L10_S2.

Applications listed:

- **Network Design:** Min cut helps in improving the efficiency of communication and optimizing network flow. The algorithm is used in network design to find the minimum capacity cut L10_S2.
- **Communication Networks:** For understanding the vulnerability of the networks to failures minimum cut can be useful. It helps building robust and fault-tolerant communication networks L10_S2.
- **VLSI Design:** In Very Large Scale Integration (VLSI) design, the algorithm is useful for partitioning circuits into smaller components leading to reduced interconnectivity complexity L10_S2.

Reference: Section 1.5, *Application: A Randomized Min-Cut Algorithm* in *Probability and Computing (2nd Edition)* L10_S2.

1.2 What is Min-Cut?

A **cut-set** in a graph is a set of edges whose removal breaks the graph into two or more connected components L10_S2.

Given a graph

$$G = (V, E)$$

with n vertices, the **minimum cut (min-cut) problem** is to find a minimum cardinality cut-set in G L10_S2.

Min-cut algorithms, such as Karger's algorithm, are random and can be sensitive to the initial choice of edges. If the algorithm happens to contract critical edges early in the process, it might find a smaller cut L10_S2.

1.3 Edge Contraction

The main operation in the algorithm is **edge contraction**, which is operation that removes an edge from a graph while simultaneously merging the two vertices L10_S2.

In contracting an edge (u, v):

1. Merge the two vertices u and v into one vertex.
2. Eliminate all edges connecting u and v.
3. Retain all other edges in the graph.
4. The new graph may have parallel edges but no self-loops L10_S2.

---

2. Successful and Unsuccessful Min-Cut Runs

2.1 Successful Min-Cut Run

A successful min cut run refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph L10_S2.

The lecture includes a figure titled "Figure: Successful Run" illustrating contractions that result in the correct minimum cut L10_S2.

2.2 Unsuccessful Min-Cut Run

An unsuccessful min-cut run refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph L10_S2.

The lecture includes a figure titled "Figure: Unsuccessful Run" illustrating contractions that lead to a non-minimum cut L10_S2.

---

3. Max-Flow Min-Cut Theorem

The max-flow min-cut theorem states:

> "In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut." L10_S2

Definitions:

- **Capacity of a cut:** The sum of the capacity of the edges in the cut that are oriented from a vertex $\in X$ to a vertex $\in Y$ L10_S2.
- **Minimum cut:** The cut in the network that has the smallest possible capacity L10_S2.
- **Minimum cut capacity:** The capacity of the minimum cut L10_S2.
- **Maximum flow:** The largest possible flow from source S to sink T L10_S2.

---

4. Deterministic Min-Cut Algorithm

4.1 Stoer–Wagner Min-Cut Algorithm

Let s and t be two vertices of a graph G. Let $G/\{s,t\}$ be the graph obtained by merging s and t.

Then a minimum cut of G can be obtained by taking the smaller of:

1. A minimum s-t-cut of G;
2. A minimum cut of $G/\{s,t\}$ L10_S2.

Reasoning stated in lecture:

- Either there is a minimum cut of G that separates s and t, then a minimum s-t-cut of G is a minimum cut of G.
- Or there is none, then a minimum cut of $G/\{s,t\}$ does the job L10_S2.

4.2 Pseudocode

Algorithm 1: MinimumCutPhase(G, a) L10_S2

1. $A \leftarrow \{a\}$
2. while $A \neq V$ do
    - add to A the most tightly connected vertex
3. return the cut weight as the "cut of the phase"

Algorithm 2: MinimumCut(G) L10_S2

1. while $|V| \geq 1$ do
    - choose any a from V;
    - MinimumCutPhase(G, a);
    - if the cut-of-the-phase is lighter than the current minimum cut then
        - store the cut-of-the-phase as the current minimum cut;
    - shrink G by merging the two vertices added last;
2. return the minimum cut

---

5. Randomized Min-Cut Algorithm

5.1 Why Randomized Algorithm?

Randomized algorithms provide a probabilistic guarantee of success. It provides a more accurate estimate of the minimum cut with fewer iterations L10_S2.

Listed characteristics:

- Efficiency
- Parallelization
- Approximation Guarantees
- Avoidance of Worst-Case Instances
- Heuristic Nature
- Robustness L10_S2.

5.2 Karger's Randomized Algorithm

The lecture includes an example run where random edges are picked in a way that the output cut is not minimal. The output cut produced is {a, c, e}, but the minimal cut is either {b, e} or {a, d} L10_S2.

This demonstrates that random contraction choices can lead to an unsuccessful run.

5.3 Pseudocode: Recursive-Randomized-Min-Cut(G, α)

Input:
An undirected multigraph G with n vertices and an integer constant $\alpha > 0$.

Output:
A cut C of G L10_S2.
If $n \leq \alpha^3$ then

- $C \leftarrow$ a min-cut of G found using brute force (exhaustive) search;

Else

- for $i \leftarrow 1$ to α do
  - $G' \leftarrow$ multigraph obtained by applying $n - \frac{n}{\sqrt{\alpha}}$ random contraction steps on G;
  - $C' \leftarrow$ Recursive-Randomized-Min-Cut(G', α);
  - if $i = 1$ or $|C'| < |C|$ then
    - $C \leftarrow C'$;

Return C L10_S2.

---

6. Comparison: Deterministic vs Randomized Min-Cut Algorithm

Any specific problem is responsible for deciding the approach L10_S2.

Deterministic Min Cut:

- It always guarantees an exact minimum cut.
- It may have higher time complexity for large graphs.
- The Stoer-Wagner algorithm has time complexity

$$O(V \cdot E + V^2 \log V)$$

  where V is the number of vertices and E is the number of edges L10_S2.

Randomized Min Cut:

- It provides an approximate minimum cut with high probability.
- Karger's algorithm has time complexity

$$O(V^2)$$

where V is the number of vertices [L10_S2].

---

7. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least

$$\frac{2}{n(n-1)}.$$

[L10_S2].

---

8. Python Simulation

- Open Campuswire Post regarding today's lecture (L10).
- Download the `.ipynb` file pasted in the comments [L10_S2].

End of Lecture 10

Sources